Name: Devasish Mishra
BITS ID: 2025eb01546

Q1). Task: Prove using natural deduction that the proposition $(p \wedge q) \vee (p \wedge r)$ can be derived from the proposition $p \wedge (q \vee r)$.

Proof:

1. $p \wedge (q \vee r)$ (Premise)
2. p ($\wedge$-elimination on 1)
3. $q \vee r$ ($\wedge$-elimination on 1)
4. Assume q (Assumption for $\vee$-elimination):
   5. $p \wedge q$ ($\wedge$-introduction on 2 and 4)
   6. $(p \wedge q) \vee (p \wedge r)$ ($\vee$-introduction on 5)
7. Assume r (Assumption for $\vee$-elimination):
   8. $p \wedge r$ ($\wedge$-introduction on 2 and 7)
   9. $(p \wedge q) \vee (p \wedge r)$ ($\vee$-introduction on 8).
10. $(p \wedge q) \vee (p \wedge r)$ ($\vee$-elimination on ~~3, 4, 6~~ 3, 4-6, and 7-9).

Name: Devasish Mishra

BITS ID: 2025eb01546

Q2). Task: Evaluate the two propositional logic formulae for correctness based on grammer, and either provide a falsifying assignment or correct the syntax.

a). $(P \rightarrow Q) \rightarrow ((R \vee S) \wedge (T \vee U))$

- Well-formed? Yes.
- Falsifying Assignment: To make ~~the~~ an implication evaluate to "False", the left side must be "True" and right side must be "False".
  - Set P = False, Q = False (This makes $P \rightarrow Q$ True)
  - Set R = False, S = False, T = False, U = False.
  - Final Assignment: P = False, Q = False, R = False, S = False, T = False, U = False.

b). $(X \wedge Y) \wedge (Z \rightarrow W) \leftrightarrow (\wedge A \wedge B)$

- Well-formed? No. The sequence $(\wedge A \wedge B)$ has a syntax error because the AND operator $(\wedge)$ is binary and requires a left operand.
- Correction: Change the first $(\wedge)$ to the NOT operator $(\neg)$ to make it grammatically correct.
  - Rewritten Formula:

$$(X \wedge Y) \wedge (Z \rightarrow W) \leftrightarrow (\neg A \wedge B)$$

Name: Devasish Mishra
BITS ID: 2025eb01546

Q3). Task: Check satisfiability of $(P \vee \neg Q) \wedge (\neg P \vee R) \wedge (\neg Q \vee R) \wedge (\neg R \vee T) \wedge (\neg S \vee T) \wedge (S \vee \neg T)$ using the DPLL algorithm.

DPLL Steps:

1. Identify Clauses:

$C_1: \{P, \neg Q\}$

$C_2: \{\neg P, R\}$

$C_3: \{\neg Q, R\}$

$C_4: \{\neg R, T\}$

$C_5: \{\neg S, T\}$

$C_6: \{S, \neg T\}$.

2. Pure Literal Elimination: Notice that $\neg Q$ is a pure literal (it only appears negatively in C_1 and C_3).
   - Action: Assign $Q = False$.
   - Result: Clauses C_1 and C_3 are satisfied and removed.

3. Pure literal Elimination: In the remaining clauses (C_2, C_4, C_5, C_6), $\neg P$ is now a pure literal (only appears in C_2).
   - Action: Assign $P = False$.
   - Result: Clause C_2 is satisfied and removed.

4. Pure literal elimination: In the remaining clauses (C_4, C_5, C_6), $\neg R$ is now a pure literal (only appers in C_4).
   - Action: Assign $R = False$.
   - Result: Clause C_4 is satisfied and removed.

5. Branching / Assignment: The remaining clauses are $C_5: \{\neg S, T\}$ and $C_6: \{S, \neg T\}$, which logically translate to $S \Leftrightarrow T$.
   - Action: Assign $S = True$ and $T = True$ (or both F).
   - Result: Clauses C_5 and C_6 are satisfied.

Conclusion: The formula is Satisfiable. The assignment is $P = False$, $Q = False$, $R = False$ (with $S = True$, $T = True$).

Name: Devansh Mishra
BITS ID: 2025eb01546

Q4). Task: Mark statements as True or False.

- a). False. In Disjunctive Normal Form (DNF), an expression is a series of conjunctions joined by ORs. If one conjunction (disjunctive clause) is false, the entire expression can still be true as long as least one other conjuction evaluates ~~ar~~ to true.
- b). True. The statement "If a number is negative (N), then its square is positive (S)" translate to $N \rightarrow S$. Its contrapositive is $\neg S \rightarrow \neg N$, which precisely ~~matched~~ matches "If a number's square is not positive, then the number is not negative." These are logically equivalent.
- c). True. We can prove semantic equivalence algebraically: $(P \vee Q) \rightarrow R$

$$\equiv \neg(P \vee Q) \vee R \quad \text{(Implication equivalence)}$$
$$\equiv (\neg P \wedge \neg Q) \vee R \quad \text{(De Morgan's Law)}$$
$$\equiv (\neg P \vee R) \wedge (\neg Q \vee R) \quad \text{(Distributive Law)}$$
$$\equiv (R \vee \neg P) \wedge (R \vee \neg Q) \quad \text{(Commutative Law)}$$
$$\equiv (\neg R \rightarrow \neg P) \wedge (\neg R \rightarrow \neg Q) \quad \text{(Implication equivalence)}.$$

- d). False. The argument presented is a classic example of Modus Ponens (If P then Q; P; therefore Q), not Modus Tollens (If P then Q; Not Q; therefore Not P).

Name : Devasish Mishra
BITS ID : 2025eb01546

R5). Task : Prove the equivalence $(P \vee Q) U (R \wedge S) \equiv (P \vee Q) W (R \wedge S) \wedge F(R \wedge S)$.

Proof using Definitions :

- Left-hand side (Until - U) : $\phi U \psi$ states that there exists some future state $k \geq 0$ where ψ holds, and for all states i prior to k $(0 \leq i \leq k)$, ϕ holds continuously.
- Right hand side (Weak Until - W AND Finally - F) :
  * $\phi W \psi$ states that either ϕ holds indefinitely forever $(G\phi)$, OR $\phi U \psi$ holds.
    - $F\psi$ states that there exists some future ~~conts~~ state where ψ is guaranteed to hold.
- Reasoning : If we evaluate $(\phi W \psi) \wedge F\psi$, the $F\psi$ condition dictates that ψ must eventually occur. Because ψ is guaranteed to occur at some point, the scenario where ϕ holds indefinitely without ψ ever occuring (the $G\phi$ portion of Weak Until) becomes impossible. Therefore, the only way for $(\phi W \psi)$ to remain true when ψ is guaranteed to happen is if ϕ holds strictly up until ψ occurs. This is the exact definition of strong Until $(\phi U \psi)$.
- Therefore, $(P \vee Q) U (R \wedge S) \equiv (P \vee Q) W (R \wedge S) \wedge F(R \wedge S)$.

Name : Devasish Mishra
BITS ID : 2025eb01546.

Q6). Based on transitions in TS3 starting from S0 : S0 → S1, S2 ; S1 → S4 ; S4 → S2 ; S2 → S3 ; S3 → S0 :

a). F¬m : True → The only state satisfying m is S0. From S0 all paths move to either S1 or S2 neither of which satisfy m.

b). G(n → Fp) : True → states satisfying n are S1 & S2. From S1, next state is S4, which satisfies p. From S2 → S3 which satisfies p. ∴ any time n occurs, p eventually follows.

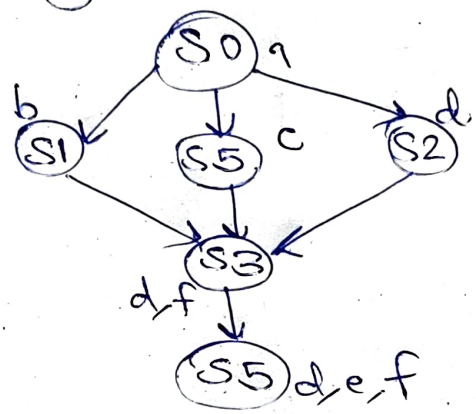
c). EGP : False → This means, eventually p becomes True & stays true forever. as all Paths go back to S0 (which ~~doesn~~ doesnt satisfy p), p will always alternate between True & False.

d). GF(n∧o) : True → State S2 is the only state satisfying both n & o. Because all possible paths through ~~the~~ the transition system must eventually funnel through S2 before looping back, ~~it~~ is visited indefinitely after on every path.

e). (m∧Fo)Up : False → The 'until' operator needs the left side to be True until right side becomes true. At S0, p is false but (m∧Fo) is true. But the system can transition to S2. At S2, p is false & (m∧Fo) is also False. (because m is false at S2). Because we hit a state where neither the left nor right side is true, the until property breaks.

Name : Devasish Mishra
BITS ID: 2025 ~~eb014~~ eb01546.

Q7). Based on TS5 starting from S0: S1 → S5 (mid), S2. And all funnels to S3, which cycles with S5 (bottom).



a). AX(AFF): True → This formula means "in all next states (AX), for all paths, f will eventually be True (AF)". From S0, next states are S1, S2, S5. From any of these states, all path lead to S3, which satisfies f. Therefore, in all next states, f is surely reached.

b). EG(EFd): True → This means "There exists a path where globally (EG), there is always a possible path to d (EFd)".

Consider path S0 → S1 → S2 → S5 → S3 .....

- At S0, EFd is True (can reach S2 or S3).
- At S1, EFd is True (can reach S3).
- At S3 & S5 (bottom): d is currently True. So EFd trivially holds.
- Because EFd holds at every state along the path, EG(EFd) is satisfied.